

## 1.1 Reading and writing data:

### 1.1.1 Data cleaning and pre-processing using Pandas and Numpy

Pandas is a popular open-source data manipulation and analysis library for Python. It provides easy-to-use functions needed to work with structured data seamlessly.

Pandas also integrates seamlessly with other popular Python libraries, such as NumPy for numerical computing and Matplotlib for data visualization. This makes it a powerful asset for data driven tasks.

Pandas excels in handling missing data, reshaping datasets, merging and joining multiple datasets, and performing complex operations on data, making it exceptionally useful for data cleaning and manipulation.

**Data cleaning** involves identifying and rectifying errors, inconsistencies, and missing values within a dataset. It's like preparing your ingredients before cooking; you want everything in order to get the perfect analysis or visualization.

**Data cleaning** is the initial phase of refining your dataset, making it readable and usable with techniques like removing duplicates, handling missing values and data type conversion while **data preprocessing** is similar to taking this refined data and scaling with more advanced techniques such as feature engineering, encoding categorical variables and handling outliers to achieve better and more advanced results.

### How to check pandas install or not

```
Python 3.9.4 (tags/v3.9.4:1f2e308, Apr 6 2021, 13:40:21) [MSC v.1928 64
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> print("vimal")
vimal
>>> 2+3
5
>>> import pandas
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    import pandas
ModuleNotFoundError: No module named 'pandas'
>>> import pandas
>>> |
```

### How to Install Pandas library

→ Write **pip install pandas** on command prompt.

```
C:\>pip install pandas
Collecting pandas
  Downloading pandas-2.2.3-cp39-cp39-win_amd64.whl (11.6 MB)
    | 11.6 MB 6.4 MB/s
Collecting python-dateutil>=2.8.2
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
    | 229 kB ...
Collecting pytz>=2020.1
  Downloading pytz-2025.2-py2.py3-none-any.whl (509 kB)
    | 509 kB 6.8 MB/s
Collecting tzdata>=2022.7
  Downloading tzdata-2025.2-py2.py3-none-any.whl (347 kB)
    | 347 kB ...
Collecting numpy>=1.22.4; python_version < "3.11"
  Downloading numpy-2.0.2-cp39-cp39-win_amd64.whl (15.9 MB)
    | 15.9 MB 6.4 MB/s
```

→ If pandas already install then following message will come

```
C:\>pip install pandas
Requirement already satisfied: pandas in c:\users\vimalvaiwala\appdata\lo
Requirement already satisfied: numpy>=1.22.4 in c:\users\vimalvaiwala\app
das) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\vimalva
(from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\vimalvaiwala\appd
as) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\vimalvaiwala\ap
ndas) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\vimalvaiwala\appdata\
ateutil>=2.8.2->pandas) (1.17.0)
```

## What is Data Cleaning and Preprocessing?

**Data Cleaning and Preprocessing** is the initial and essential step in any data analysis or machine learning pipeline. It ensures that the data is:

- ✓ Accurate
- ✓ Consistent
- ✓ Complete
- ✓ Ready for analysis or modeling

## Why is it Necessary?

Raw data often contains:

- **Missing values** (e.g., no age entered)
- **Inconsistent values** (e.g., "Male", "male", "MALE")
- **Duplicate entries**
- **Wrong formats** (e.g., numbers as strings)
- **Outliers or incorrect values**

**Table of Row Data**

| Index | Name   | Gender | Age  | Marks | Passed |
|-------|--------|--------|------|-------|--------|
| 0     | Raj    | Male   | 18.0 | 85.0  | Yes    |
| 1     | Priya  | Female | 19.0 | 92.0  | Yes    |
| 2     | Vikram | male   | NaN  | NaN   | No     |
| 3     | Neha   | FEMALE | 21.0 | 77.0  | NaN    |
| 4     | NaN    | Male   | 20.0 | 85.0  | yes    |

**Step 1: Create Raw Data**

```
import pandas as pd
import numpy as np

data = {
    'Name': ['Raj', 'Priya', 'Vikram', 'Neha', np.nan],
    'Gender': ['Male', 'Female', 'male', 'FEMALE', 'Male'],
    'Age': [18, 19, np.nan, 21, 20],
    'Marks': [85, 92, np.nan, 77, 85],
    'Passed': ['Yes', 'Yes', 'No', np.nan, 'yes']
}

df = pd.DataFrame(data)
print("◆ Raw Data:\n", df)
```

**Step 2 :Step-by-Step Data Cleaning**  
 Step 2: Handle Missing Values

```
# Check how many missing values
print(df.isnull().sum())
```

You'll find that:

- Name, Age, Marks, and Passed have missing (NaN) values.

👉 Solutions:

- **Drop** rows where essential data is missing (like Name).
- **Fill** numerical missing values with mean/median.
- **Fill** categorical missing values with mode.

```
# Drop rows where Name is missing
df.dropna(subset=['Name'], inplace=True)
```

```
# Fill missing Age with mean
df['Age'].fillna(df['Age'].mean(), inplace=True)
```

```
# Fill missing Marks with median
df['Marks'].fillna(df['Marks'].median(), inplace=True)
```

**# Fill missing Passed with most common value**

```
df['Passed'].fillna(df['Passed'].mode()[0], inplace=True)
```

**Step 3: Fix Inconsistent Categories**

Observe how "Gender" and "Passed" are written in different cases.

```
df['Gender'] = df['Gender'].str.lower().str.capitalize() # Male, Female
```

```
df['Passed'] = df['Passed'].str.lower().str.capitalize() # Yes, No
```

**Step 4: Remove Duplicates**

```
df.drop_duplicates(inplace=True)
```

**Step 5: Convert Data Types**

Ensure Age is an integer:

```
df['Age'] = df['Age'].astype(int)
```

**Step 6: Encode Categorical Variables**

Convert Gender and Passed into numeric format for analysis or modeling:

```
df['Gender_Code'] = df['Gender'].map({'Male': 1, 'Female': 0})
```

```
df['Passed_Code'] = df['Passed'].map({'Yes': 1, 'No': 0})
```

**Data transformation and normalization using Numpy**

**Data Transformation** is the process of **modifying, converting, or restructuring** raw data into a format that is more suitable for **analysis, processing, or visualization**.

This transformation is an essential step in:

- **Data preprocessing**
- **Data cleaning**
- **Feature engineering** (in machine learning)

**Why is Data Transformation Important?**

- Raw data is often **inconsistent, unclean, or scattered**.
- Different columns (features) may be in different **scales or formats**.
- Models and tools require **clean, consistent, and numeric** data.

Let's take a small, realistic dataset from a **student information system**:

| Name   | Gender | Age | Marks (%) | Passed |
|--------|--------|-----|-----------|--------|
| Raj    | Male   | 18  | 85        | Yes    |
| Priya  | Female | 19  | 92        | Yes    |
| Vikram | Male   | 20  | 54        | No     |
| Neha   | Female | 21  | 77        | Yes    |

**Transformation Examples**

Most algorithms cannot process text (like "Male", "Yes"). We convert it to numbers.

| Gender | Passed |
|--------|--------|
| Male   | Yes    |
| Female | No     |

→ After **Label Encoding**:

| Gender | Passed |
|--------|--------|
| 1      | 1      |
| 0      | 0      |

Now the data is numerical and usable by ML algorithms.

### Scaling / Normalization (Bringing values to common range)

Let's scale "Marks (%)" using **Min-Max Normalization**:

$$\text{Normalized Marks} = \frac{(x - \min)}{(\max - \min)}$$

Original Marks: [85, 92, 54, 77]

Min = 54, Max = 92

$$\text{Normalized 85} = \frac{85 - 54}{92 - 54} = \frac{31}{38} \approx 0.8158$$

| Name   | Original Marks | Normalized Marks |
|--------|----------------|------------------|
| Raj    | 85             | 0.8158           |
| Priya  | 92             | 1.0000           |
| Vikram | 54             | 0.0000           |
| Neha   | 77             | 0.6053           |

### Aggregation (Summarizing data)

Let's group data by **Gender** and find **average marks**:

| Gender | Avg Marks        |
|--------|------------------|
| Male   | (85+54)/2 = 69.5 |
| Female | (92+77)/2 = 84.5 |

### Binning (Range-based transformation)

| Age | Age Group |
|-----|-----------|
| 18  | Teenager  |
| 19  | Teenager  |

| Age | Age Group |
|-----|-----------|
| 20  | Adult     |
| 21  | Adult     |

### Final Transformed Data Table (Example)

| Name   | Gender (0/1) | Age Group | Marks (0–1) | Passed (0/1) |
|--------|--------------|-----------|-------------|--------------|
| Raj    | 1            | Teenager  | 0.8158      | 1            |
| Priya  | 0            | Teenager  | 1.0000      | 1            |
| Vikram | 1            | Adult     | 0.0000      | 0            |
| Neha   | 0            | Adult     | 0.6053      | 1            |

### Summary

| Transformation Type | Purpose                           | Real Example             |
|---------------------|-----------------------------------|--------------------------|
| Encoding            | Convert categories to numbers     | "Male" → 1, "Female" → 0 |
| Scaling             | Normalize range of numeric values | Marks → 0 to 1           |
| Aggregation         | Group and summarize data          | Average marks by gender  |
| Binning             | Convert continuous to categorical | Age → Teenager, Adult    |

**"Data transformation" bridges the gap between raw information and meaningful insights.** Without it, analysis and machine learning become unreliable or even impossible.

To install sklearn.preprocessing, used following command:  
→ pip install scikit-learn

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
```

# Step 1: Original Raw Data

```
data = {
    'Name': ['Raj', 'Priya', 'Vikram', 'Neha'],
    'Gender': ['Male', 'Female', 'Male', 'Female'],
    'Age': [18, 19, 20, 21],
    'Marks': [85, 92, 54, 77],
    'Passed': ['Yes', 'Yes', 'No', 'Yes']
}
```

```
df = pd.DataFrame(data)
```

**OR**

```
data = pd.read_csv('student_data.csv')
df = pd.DataFrame(data)
```

```

print("Original Data:\n", df)

# Step 2: Label Encoding for 'Gender' and 'Passed'
le_gender = LabelEncoder()
le_passed = LabelEncoder()
df['Gender_Encoded'] = le_gender.fit_transform(df['Gender']) # Male = 1,
Female = 0
df['Passed_Encoded'] = le_passed.fit_transform(df['Passed']) # Yes = 1, No = 0

# Step 3: Min-Max Normalization for Marks
scaler = MinMaxScaler()
df['Marks_Normalized'] = scaler.fit_transform(df[['Marks']])

# Step 4: Binning Age into Age Group
def age_group(age):
    if age <= 19:
        return 'Teenager'
    else:
        return 'Adult'

df['Age_Group'] = df['Age'].apply(age_group)

# Step 5: Aggregation Example: Average Marks by Gender
agg_df =
df.groupby('Gender')['Marks'].mean().reset_index().rename(columns={'Marks':
'Avg_Marks'})

# Display Results
print("\nTransformed Data:\n", df[['Name', 'Gender_Encoded', 'Age_Group',
'Marks_Normalized', 'Passed_Encoded']])
print("\n Aggregated Data (Avg Marks by Gender):\n", agg_df)

```

## Exploratory Data Analysis

**Exploratory Data Analysis (EDA)** is the process of examining a dataset to summarize its main characteristics, often using visual methods. It helps to:

- Understand the structure of the data
- Detect outliers and anomalies
- Discover patterns and relationships
- Prepare for modeling (e.g., selecting features)

EDA is like "**getting to know your data**" before doing any predictive analysis or machine learning.

### Steps in EDA

#### 1. Understand the Data Types

- Categorical (e.g., Gender: Male/Female)

- Numerical (e.g., Age, Salary)
- Date/Time
- 2. **Check for Missing Values**
- 3. **Basic Statistics**
  - Mean, Median, Mode, Standard Deviation
- 4. **Data Visualization**
  - Histograms (distribution)
  - Boxplots (outliers)
  - Bar charts (categorical data)
  - Correlation Heatmaps (relationships)
- 5. **Outlier Detection**

EDA in Action

#### 1. Data Summary

```
df.describe()
```

Gives you: Mean, Min, Max, Std. Deviation of Age and Marks.

#### 2. Missing Values Check

```
df.isnull().sum()
```

See if any values are missing in Age, Marks, or Gender.

#### 3. Gender Distribution

```
df['Gender'].value_counts().plot(kind='bar')
```

Gives you how many males and females are there.

#### 4. Marks Distribution

```
df['Marks'].plot(kind='hist', bins=5)
```

See how marks are distributed: high scores, low scores, etc.

#### 5. Boxplot to Check Outliers

```
import seaborn as sns
sns.boxplot(data=df, y='Marks')
```

If Vikram has much lower marks than others, the boxplot will highlight him as an outlier.

#### 6. Correlation Check

```
df.corr()
```

#### Outcome of EDA

- Detect students with unusually high/low marks
- Understand performance by gender
- See if age affects marks



- Prepare data for further steps like classification or regression

**Example**

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Step 2: Create the dataset
data = {
    'Student': ['Vimal', 'Priya', 'Vikram', 'Neha', 'Aarav'],
    'Gender': ['Male', 'Female', 'Male', 'Female', 'Male'],
    'Age': [18, 19, 18, 21, 20],
    'Marks': [85, 92, 45, 77, 85]
}

df = pd.DataFrame(data)

# Step 3: Display the first few rows
print("\n📊 Dataset Preview:")
print(df.head())

# Step 4: Summary statistics
print("\n📈 Descriptive Statistics:")
print(df.describe())

# Step 5: Check for missing values
print("\n🔍 Missing Values:")
print(df.isnull().sum())

# Step 6: Gender Distribution
plt.figure(figsize=(5, 4))
sns.countplot(x='Gender', data=df)
plt.title("Gender Distribution")
plt.show()

# Step 7: Marks Distribution
plt.figure(figsize=(6, 4))
plt.hist(df['Marks'], bins=5, color='skyblue', edgecolor='black')
plt.title("Marks Distribution")
plt.xlabel("Marks")
plt.ylabel("Number of Students")
plt.show()

# Step 8: Boxplot for Outlier Detection
plt.figure(figsize=(4, 5))
sns.boxplot(y=df['Marks'], color='lightgreen')
plt.title("Marks Boxplot")
plt.show()
```

```
# Step 9: Correlation Heatmap
plt.figure(figsize=(5, 4))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
```

### Data visualization using Matplotlib and Seaborn

Data visualization is an easier way of presenting the data, however complex it is, to analyze trends and relationships amongst variables with the help of pictorial representation.

The following are the advantages of Data Visualization

- Easier representation of complex data
- Highlights good and bad performing areas
- Explores relationship between data points
- Identifies data patterns even for larger data points

### Matplotlib

It is an amazing visualization library in Python for 2D plots of arrays. It is a multi-platform data visualization library built on *NumPy* arrays and designed to work with the broader *SciPy* stack. It was introduced by John Hunter in the year 2002. Let's try to understand some of the benefits and features of *matplotlib*

- It's fast, efficient as it is based on *numpy* and also easier to build
- Has undergone a lot of improvements from the open source community since inception and hence a better library having advanced features as well
- Well maintained visualization output with high quality graphics draws a lot of users to it
- Basic as well as advanced charts could be very easily built
- From the users/developers point of view, since it has a large community support, resolving issues and debugging becomes much easier

### Seaborn

Conceptualized and built originally at the Stanford University, this library sits on top of *matplotlib*. In a sense, it has some flavors of *matplotlib* while from the visualization point, it is much better than *matplotlib* and has added features as well. Below are its advantages

- Built-in themes aid better visualization
- Statistical functions aiding better data insights
- Better aesthetics and built-in plots
- Helpful documentation with effective examples

## 1. Matplotlib Visualizations

Matplotlib is a basic but powerful plotting library in Python.

### a. Bar Chart – Marks of Each Student

```
import matplotlib.pyplot as plt
plt.figure(figsize=(7, 4))
plt.bar(df['Student'], df['Marks'], color='skyblue')
plt.title('Student Marks')
plt.xlabel('Student')
plt.ylabel('Marks')
plt.grid(axis='y', linestyle='--')
plt.show()
```

---

### b. Line Chart – Age vs Marks

```
plt.plot(df['Age'], df['Marks'], marker='o', color='green')
plt.title('Age vs Marks')
plt.xlabel('Age')
plt.ylabel('Marks')
plt.grid(True)
plt.show()
```

---

### c. Histogram – Distribution of Marks

```
plt.hist(df['Marks'], bins=5, color='orange', edgecolor='black')
plt.title('Histogram of Marks')
plt.xlabel('Marks')
plt.ylabel('Frequency')
plt.show()
```

## 2. Seaborn Visualizations

Seaborn is built on top of Matplotlib and provides beautiful default styles.

```
import seaborn as sns
```

---

### a. Count Plot – Gender Distribution

```
sns.countplot(x='Gender', data=df, palette='pastel')
plt.title('Gender Count')
plt.show()
```

---

### b. Boxplot – To Find Outliers in Marks

```
sns.boxplot(y='Marks', data=df, palette='Set2')
plt.title('Boxplot of Marks')
plt.show()
```

### c. Scatter Plot – Age vs Marks

```
sns.scatterplot(x='Age', y='Marks', hue='Gender', data=df, s=100)
plt.title('Scatter Plot: Age vs Marks')
plt.show()
```

### d. Heatmap – Correlation

```
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

| Plot Type    | Matplotlib             | Seaborn                    | Purpose                             |
|--------------|------------------------|----------------------------|-------------------------------------|
| Bar Chart    | <code>bar()</code>     | <code>barplot()</code>     | Compare values (e.g. marks)         |
| Line Chart   | <code>plot()</code>    | ✗                          | Show trends                         |
| Histogram    | <code>hist()</code>    | <code>histplot()</code>    | Show distribution                   |
| Boxplot      | ✗                      | <code>boxplot()</code>     | Spot outliers                       |
| Count Plot   | ✗                      | <code>countplot()</code>   | Count categories                    |
| Scatter Plot | <code>scatter()</code> | <code>scatterplot()</code> | Show relationships                  |
| Heatmap      | ✗                      | <code>heatmap()</code>     | Show correlations between variables |

### Summary Statistics Table

| Feature | Count | Unique | Top  | Mean | Std Dev | Min | 25% | 50% | 75% | Max |
|---------|-------|--------|------|------|---------|-----|-----|-----|-----|-----|
| Student | 5     | 5      | Raj  | —    | —       | —   | —   | —   | —   | —   |
| Gender  | 5     | 2      | Male | —    | —       | —   | —   | —   | —   | —   |
| Age     | 5     | —      | —    | 19.2 | 1.30    | 18  | 18  | 19  | 20  | 21  |
| Marks   | 5     | —      | —    | 76.8 | 18.55   | 45  | 77  | 85  | 85  | 92  |

### Descriptive Visual Analysis

- Distribution of Marks with KDE**
  - Shows how marks are spread; peak around 85.
  - KDE (Kernel Density Estimate) smooths the distribution curve.
- Distribution of Age with KDE**
  - Most students are around age 18–21.
  - Slight concentration around 19.
- Boxplot of Marks by Gender**
  - Highlights differences in marks distribution across male and female students.
  - Helps spot outliers (e.g., Vikram's 45).
- Violin Plot of Marks by Gender**
  - Combines KDE and boxplot.

- Gives a deeper view into the shape and spread of the data by gender.

These visuals and statistics give you a clear picture of **data spread, central tendency, and gender-based variations**, which are essential before applying any advanced analysis or modeling.

## Outlier detection and treatment

### What is an Outlier?

An **outlier** is a data point that differs significantly from other observations. It may be due to variability in data or errors.

#### Example:

If most students score between 70 and 90, but one scores **15**, it's likely an outlier.

### Why Outliers Matter?

- They can **skew statistical analysis** (mean, standard deviation).
- Can **affect machine learning models** badly.
- Sometimes outliers are **important** (e.g., fraud detection, rare diseases).

### Example Dataset: Student Marks

python

CopyEdit

```
import pandas as pd
```

```
data = {
    'Student': ['Raj', 'Priya', 'Vikram', 'Neha', 'Aarav', 'Ananya'],
    'Marks':   [85, 92, 45, 77, 85, 15] # 15 is a suspected outlier
}
```

```
df = pd.DataFrame(data)
```

### Step 1: Detecting Outliers

#### Method 1: Using Boxplot (Visual)

```
import seaborn as sns
sns.boxplot(y=df['Marks'])
```

 Points beyond the whiskers are potential outliers.

#### Method 2: Using IQR (Interquartile Range)

```
Q1 = df['Marks'].quantile(0.25)
Q3 = df['Marks'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
```

```
outliers = df[(df['Marks'] < lower_bound) | (df['Marks'] > upper_bound)]
print(outliers)
```

👉 This will identify marks like **15** or extremely high scores as outliers.

### ✂ Step 2: Treating Outliers

#### ✓ Option 1: Remove Outliers

```
df_clean = df[(df['Marks'] >= lower_bound) & (df['Marks'] <= upper_bound)]
```

Best when outliers are due to errors or irrelevant.

#### 📦 Option 2: Replace with Median

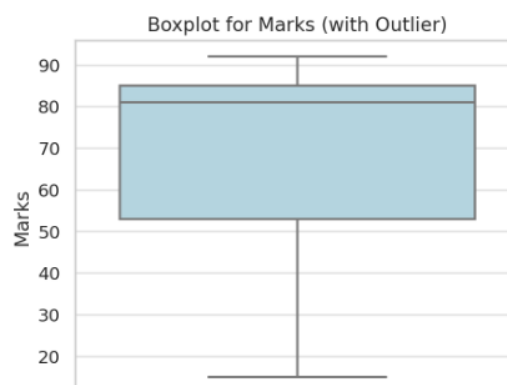
```
median_value = df['Marks'].median()
df['Marks'] = df['Marks'].apply(lambda x: median_value if x < lower_bound or
x > upper_bound else x)
```

Good when the outlier is not an error but you want to **minimize its effect**.

#### 📌 Option 3: Cap or Floor Values (Winsorizing)

```
df['Marks'] = df['Marks'].apply(lambda x: lower_bound if x < lower_bound else
upper_bound if x > upper_bound else x)
```

Restricts values within a range.



```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Step 1: Create dataset
data = {
    'Student': ['Raj', 'Priya', 'Vikram', 'Neha', 'Aarav', 'Ananya'],
    'Marks':   [85, 92, 45, 77, 85, 15] # 15 is a suspected outlier
}
```

```
df = pd.DataFrame(data)

# Step 2: Visualize using Boxplot
plt.figure(figsize=(5, 4))
sns.boxplot(y=df['Marks'], color='lightblue')
plt.title("Boxplot for Marks (with suspected outlier)")
plt.show()

# Step 3: Calculate IQR
Q1 = df['Marks'].quantile(0.25)
Q3 = df['Marks'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Step 4: Detect Outliers
outliers = df[(df['Marks'] < lower_bound) | (df['Marks'] > upper_bound)]
print("Detected Outliers:\n", outliers)

# Step 5: Treat outliers (Option 1 - Replace with median)
median_marks = df['Marks'].median()
df_treated = df.copy()
df_treated['Marks'] = df_treated['Marks'].apply(
    lambda x: median_marks if x < lower_bound or x > upper_bound else x
)

print("\nTreated DataFrame:\n", df_treated)

# Optional: Boxplot after treatment
plt.figure(figsize=(5, 4))
sns.boxplot(y=df_treated['Marks'], color='lightgreen')
plt.title("Boxplot after Outlier Treatment")
plt.show()
```

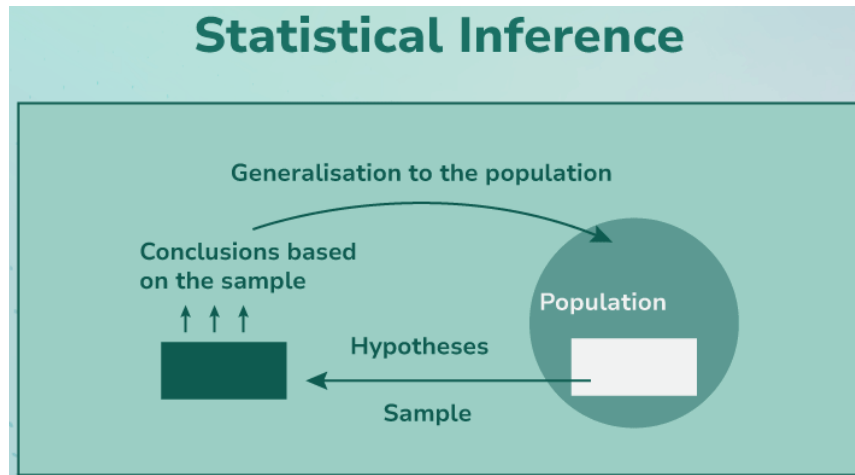
## Data Modelling and Analysis

**Data Modelling and Analysis** involves applying mathematical models to understand data patterns and make predictions or decisions. It turns raw data into useful insights using statistical techniques. Data Modelling and Analysis refers to the techniques and processes used to **understand the structure of data**, and then apply **mathematical or statistical models** to extract insights, make decisions, or predict outcomes. It includes:

1. Understanding relationships between variables
2. Estimating outcomes based on patterns
3. Making statistical inferences
4. Forecasting future trends

### Introduction to statistical models and inference

Statistical inference is the process of using data analysis to infer properties of an underlying distribution of a population. It is a branch of statistics that deals with making inferences about a population based on data from a sample.



Statistical inference is the process of drawing conclusions or making predictions about a population based on data collected from a sample of that population. It involves using statistical methods to analyze sample data and make inferences or predictions about parameters or characteristics of the entire population from which the sample was drawn.

### Branches of Statistical Inference

There are two main branches of statistical inference:

- Parameter Estimation
- Hypothesis Testing

#### Parameter Estimation

Parameter estimation is another primary goal of statistical inference. Parameters are capable of being deduced; they are quantified traits or properties related to the population you are studying. Some instances comprise the population mean, population variance, and so on-the-list. Imagine measuring each person in a town to realize the mean. This is a daunting if not an impossible task. Thus, most of the time, we use estimates.

There are two broad methods of parameter estimation:

- **Point Estimation**
- **Interval Estimation**

#### Hypothesis Testing

Hypothesis testing is used to make decisions or draw conclusions about a population based on sample data. It involves formulating a hypothesis about the population parameter, collecting sample data, and then using statistical methods to determine whether the data provide enough evidence to reject or fail to reject the hypothesis.

### Statistical Inference Methods

There are various methods of statistical inference, some of these methods are:

- Parametric Methods



- Non-parametric Methods
- Bayesian Methods

Let's discuss these methods in detail as follows:

### Parametric Methods

In this scenario, the parametric statistical methods will assume that the data is drawn from a population characterized by a probability distribution. It is mainly believed that they follow a normal distribution thus can allow one to make guesses about the populace in question. For example, the t-tests and ANOVA are parametric tests that give accurate results with the assumption that the data ought to be

- **Example:** A psychologist may ask himself if there is a measurable difference, on average, between the IQ scores of women and men. To test his theory, he draws samples from each group and assumes they are both normally distributed. He can opt for a parametric test such as t-test and assess if the mean disparity is statistically significant.

### Non-Parametric Methods

These are less assumptive and more flexible analysis methods when dealing with data out of normal distribution. They are also used to conduct data analysis when one is uncertain about meeting the assumption for parametric methods and when one have less or inadequate data. Some of the non-parametric tests include Wilcoxon signed-rank test and Kruskal-Wallis test among others.

- **Example:** A biologist has collected data on plant health in an ordinal variable but since it is only a small sample and normal assumption is not met, the biologist can use Kruskal-Wallis testing.

### *Components of a Statistical Model:*

1. **Data** – Real-world observations.
2. **Variables** – Independent (input) and dependent (output).
3. **Parameters** – Unknown constants estimated from data.
4. **Error Term** – Represents randomness or noise.

### Linear Regression Model

Linear regression is a type of **supervised machine-learning algorithm** that learns from the labelled datasets and maps the data points with most optimized linear functions which can be used for prediction on new datasets. It assumes that there is a linear relationship between the input and output, meaning the output changes at a constant rate as the input changes. This relationship is represented by a straight line.

**For example** we want to predict a student's exam score based on how many hours they studied. We observe that as students study more hours, their scores go up. In the example of predicting exam scores based on hours studied. Here

- **Independent variable (input):** Hours studied because it's the factor we control or observe.

- **Dependent variable (output):** Exam score because it depends on how many hours were studied.

### Why Linear Regression is Important?

Here's why linear regression is important:

- **Simplicity and Interpretability:** It's easy to understand and interpret, making it a starting point for learning about machine learning.
- **Predictive Ability:** Helps predict future outcomes based on past data, making it useful in various fields like finance, healthcare and marketing.
- **Basis for Other Models:** Many advanced algorithms, like logistic regression or neural networks, build on the concepts of linear regression.
- **Efficiency:** It's computationally efficient and works well for problems with a linear relationship.
- **Widely Used:** It's one of the most widely used techniques in both statistics and machine learning for regression tasks.
- **Analysis:** It provides insights into relationships between variables (e.g., how much one variable influences another).

For simple linear regression (with one independent variable), the best-fit line is represented by the equation

$$y=mx+b$$

Where:

- **y** is the predicted value (dependent variable)
- **x** is the input (independent variable)
- **m** is the slope of the line (how much y changes when x changes)
- **b** is the intercept (the value of y when x = 0)

### Example

| Diameter(X)<br>In Inches                                                               | Price(Y)<br>In Dollars                                                                 | Mean(X) | Mean(Y) | Deviations(X) | Deviations(Y) | Product of<br>Deviations | Sum of Product<br>of Deviations | Square of<br>Deviations<br>for X |
|----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|---------|---------|---------------|---------------|--------------------------|---------------------------------|----------------------------------|
| 8   | 10  | 10      | 13      | -2            | -3            | 6                        | 12                              | 4                                |
| 10  | 13  |         |         | 0             | 0             | 0                        |                                 | 0                                |
| 12  | 16  |         |         | 2             | 3             | 6                        |                                 | 4                                |

### Practical Example

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

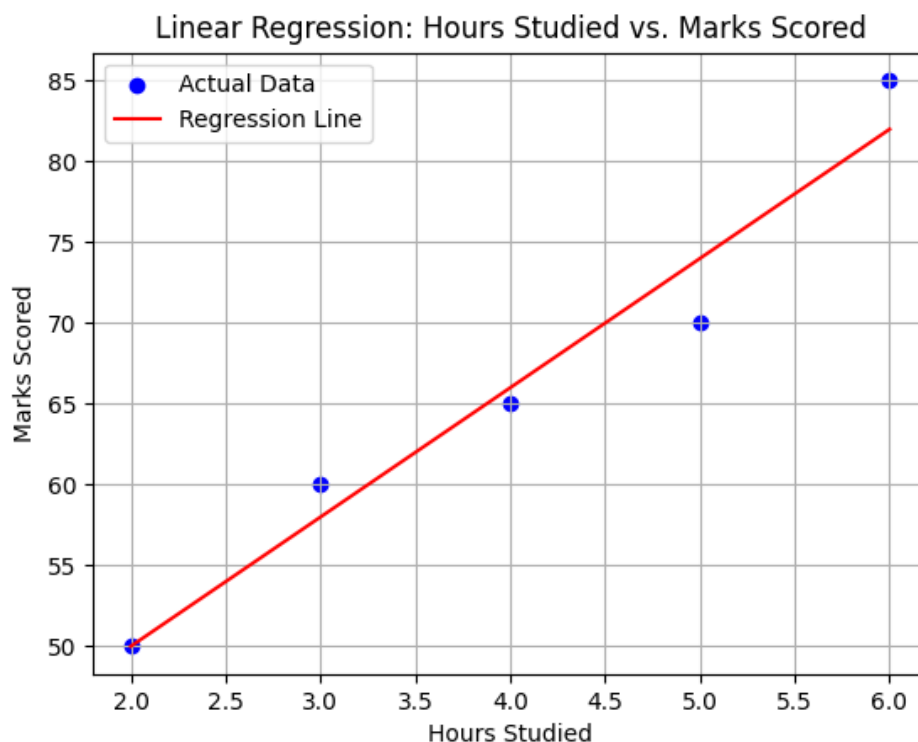
# Step 2: Create sample data (Hours Studied vs. Marks Scored)
data = {
```

```
'Hours_Studied': [2, 3, 4, 5, 6],
'Marks_Scored': [50, 60, 65, 70, 85]
}
df = pd.DataFrame(data)

# Step 3: Define X (independent) and y (dependent)
X = df[['Hours_Studied']]
Y = df[['Marks_Scored']]

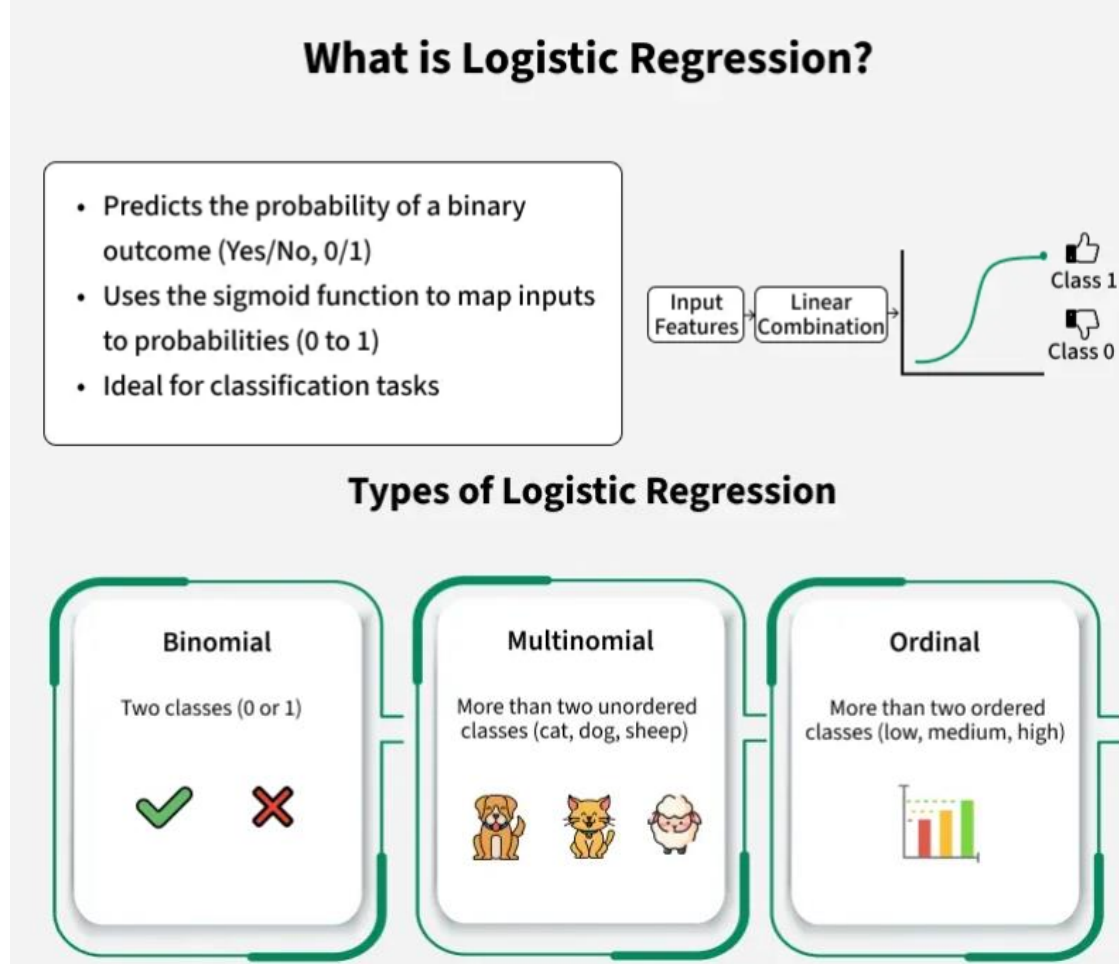
model=LinearRegression()
model.fit(X,Y)
#Prediction

'''prediction=model.predict([[10]])
print(prediction)'''
y=model.predict(X)
# Step 8: Plotting the regression line
plt.scatter(df['Hours_Studied'], df['Marks_Scored'], color='blue', label='Actual Data')
plt.plot(df['Hours_Studied'], model.predict(X), color='red', label='Regression Line')
plt.xlabel('Hours Studied')
plt.ylabel('Marks Scored')
plt.title('Linear Regression: Hours Studied vs. Marks Scored')
plt.legend()
plt.grid(True)
plt.show()
```



## Logistic Regression

Logistic Regression is a supervised machine learning algorithm used for classification problems. Unlike linear regression which predicts continuous values it predicts the probability that an input belongs to a specific class. It is used for binary classification where the output can be one of two possible categories such as Yes/No, True/False or 0/1. It uses sigmoid function to convert inputs into a probability value between 0 and 1. In this article, we will see the basics of logistic regression and its core concepts.

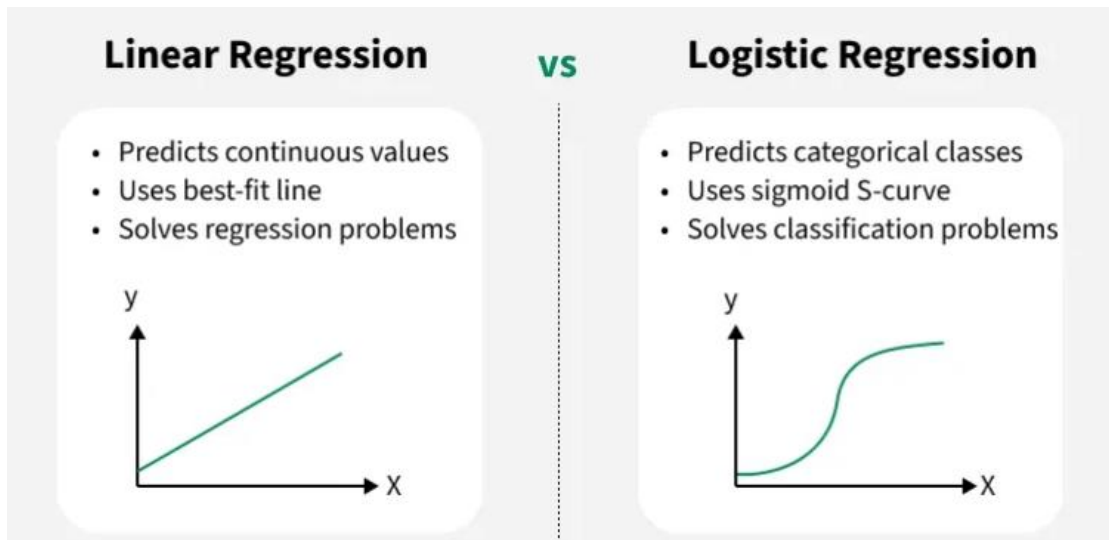


## Types of Logistic Regression

Logistic regression can be classified into three main types based on the nature of the dependent variable:

1. **Binomial Logistic Regression:** This type is used when the dependent variable has only two possible categories. Examples include Yes/No, Pass/Fail or 0/1. It is the most common form of logistic regression and is used for binary classification problems.
2. **Multinomial Logistic Regression:** This is used when the dependent variable has three or more possible categories that are not ordered. For example, classifying animals into categories like "cat," "dog" or "sheep." It extends the binary logistic regression to handle multiple classes.
3. **Ordinal Logistic Regression:** This type applies when the dependent variable has three or more categories with a natural order or ranking. Examples include ratings like "low," "medium"

and "high." It takes the order of the categories into account when modeling.



### Example

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression

# Sample data
data = {
    'Hours_Studied': [1, 2, 3, 4, 5, 6],
    'Pass': [0, 0, 0, 1, 1, 1] # 0 = Fail, 1 = Pass
}
df = pd.DataFrame(data)

# Features and target
X = df[['Hours_Studied']]
y = df['Pass']

# Train logistic regression model
model = LogisticRegression()
model.fit(X, y)

# Predict probability and class for 3.5 hours
hours_input = 3.5
predicted_prob = model.predict_proba([[hours_input]])[0][1]
predicted_class = model.predict([[hours_input]])[0]

print(f"Predicted Probability of Passing: {predicted_prob:.2f}")
print(f"Predicted Class: {'Pass' if predicted_class == 1 else 'Fail'}")

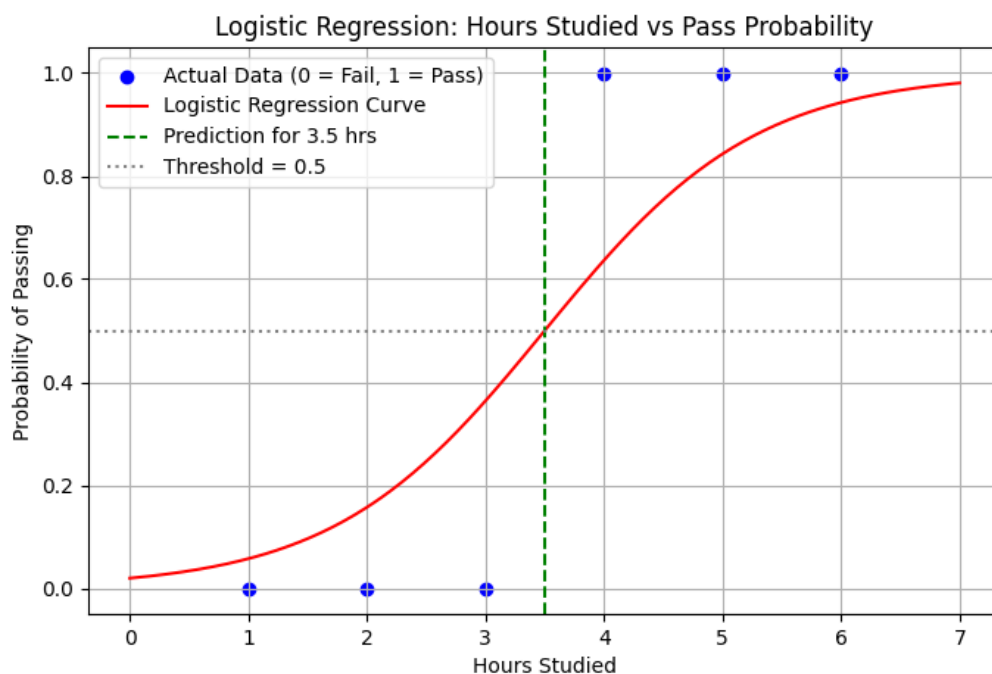
# Create a range of study hours for plotting sigmoid
```

```

x_range = np.linspace(0, 7, 300).reshape(-1, 1)
probabilities = model.predict_proba(x_range)[: , 1]

# Plot
plt.figure(figsize=(8, 5))
plt.scatter(df['Hours_Studied'], df['Pass'], color='blue', label='Actual Data
(0 = Fail, 1 = Pass)')
plt.plot(x_range, probabilities, color='red', label='Logistic Regression Curve
')
plt.axvline(x=hours_input, color='green', linestyle='--
', label=f'Prediction for {hours_input} hrs')
plt.axhline(y=0.5, color='gray', linestyle=':', label='Threshold = 0.5')
plt.title('Logistic Regression: Hours Studied vs Pass Probability')
plt.xlabel('Hours Studied')
plt.ylabel('Probability of Passing')
plt.legend()
plt.grid(True)
plt.show()

```



- ☐ **Blue dots:** Actual data points (1 = Pass, 0 = Fail)
- ☐ **Red curve:** Predicted probability from logistic regression (sigmoid shape)
- ☐ **Green dashed line:** Where the student studied 3.5 hours
- ☐ **Gray dotted line:** Threshold at probability 0.5 (decision boundary)

## Time Series analysis

**Time series data** consists of sequential data points recorded over time which is used in industries like finance, pharmaceuticals, social media and research. Analyzing and visualizing this data helps us to find trends and seasonal patterns for forecasting and decision-making. In this article, we will see more about Time Series Analysis and Visualization in depth.

### What is Time Series Data Analysis?

Time series data analysis involves studying data points collected in chronological time order to identify current trends, patterns and other behaviors. This helps extract actionable insights and supports accurate forecasting and decision-making.

#### Key Concepts in Time Series Analysis

- **Trend:** It represents the general direction in which a time series is moving over an extended period. It checks whether the values are increasing, decreasing or staying relatively constant.
- **Seasonality:** Seasonality refers to repetitive patterns or cycles that occur at regular intervals within a time series corresponding to specific time units like days, weeks, months or seasons.
- **Moving average:** It is used to smooth out short-term fluctuations and highlight longer-term trends or patterns in the data.
- **Noise:** It represents the irregular and unpredictable components in a time series that do not follow a pattern.
- **Differencing:** It is used to make the difference in values of a specified interval. By default it's 1 but we can specify different values for plots.
- **Stationarity:** A stationary time series is statistical properties such as mean, variance and autocorrelation remain constant over time.
- **Order:** The order of differencing refers to the number of times the time series data needs to be differenced to achieve stationarity.
- **Autocorrelation:** [Autocorrelation](#) is a statistical method used in time series analysis to quantify the degree of similarity between a time series and a lagged version of itself.
- **Resampling:** [Resampling](#) is a technique in time series analysis that is used for changing the frequency of the data observations.

### Types of Time Series Data

Time series data can be classified into two sections:

1. **Continuous Time Series:** Data recorded at regular intervals with a continuous range of values like temperature, stock prices, Sensor Data, etc.
2. **Discrete Time Series:** Data with distinct values or categories recorded at specific time points like counts of events, categorical statuses, etc.

### Example

```
import pandas as pd
import matplotlib.pyplot as plt

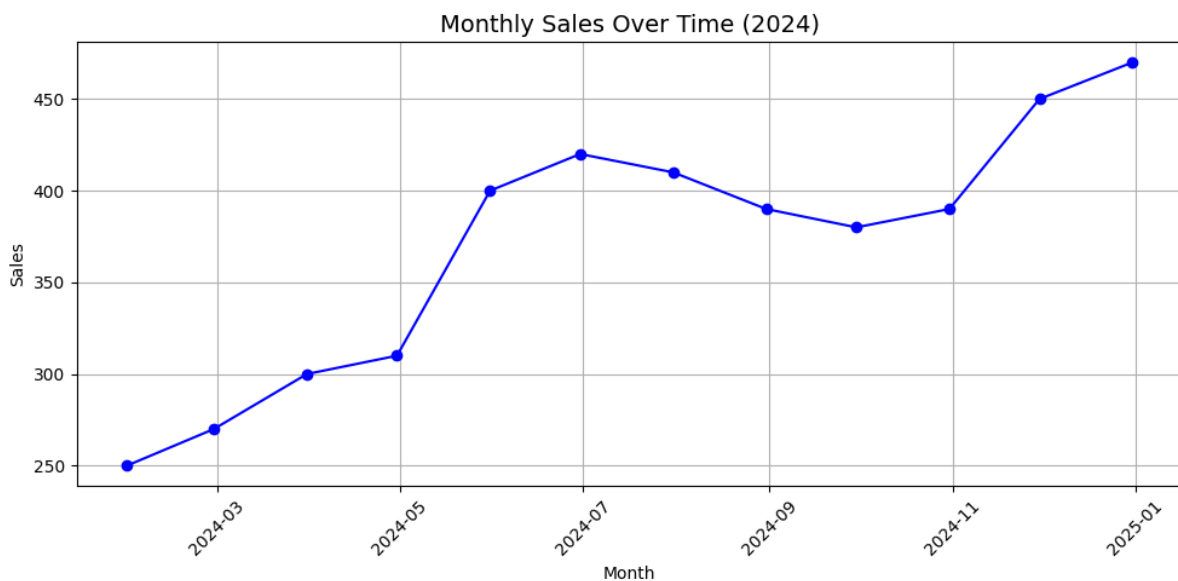
# Sample monthly sales data (Jan to Dec)
```



```
data = {
    'Month': pd.date_range(start='2024-01-01', periods=12, freq='M'),
    'Sales': [250, 270, 300, 310, 400, 420, 410, 390, 380, 390, 450, 470]
}

df = pd.DataFrame(data)
df.set_index('Month', inplace=True)

# Plotting the Time Series
plt.figure(figsize=(10, 5))
plt.plot(df.index, df['Sales'], marker='o', linestyle='-', color='blue')
plt.title('Monthly Sales Over Time (2024)', fontsize=14)
plt.xlabel('Month')
plt.ylabel('Sales')
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



- A **line plot** showing how sales vary month-by-month.
- You may observe:
  - **Trend:** Gradual increase in sales over the year.
  - **Seasonality:** If certain months (like Nov–Dec) always have higher sales, it indicates seasonal effect.

### Applications of Time Series Analysis

- Forecasting **sales**, **temperature**, or **traffic**
- Detecting **faults** or **anomalies** in sensors or machines
- Monitoring **stock markets**
- **Econometric modeling** in research



## What is Smoothing?

Smoothing is used in time series to:

- Eliminate random variation (noise)
- Highlight **underlying trends** or **patterns**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

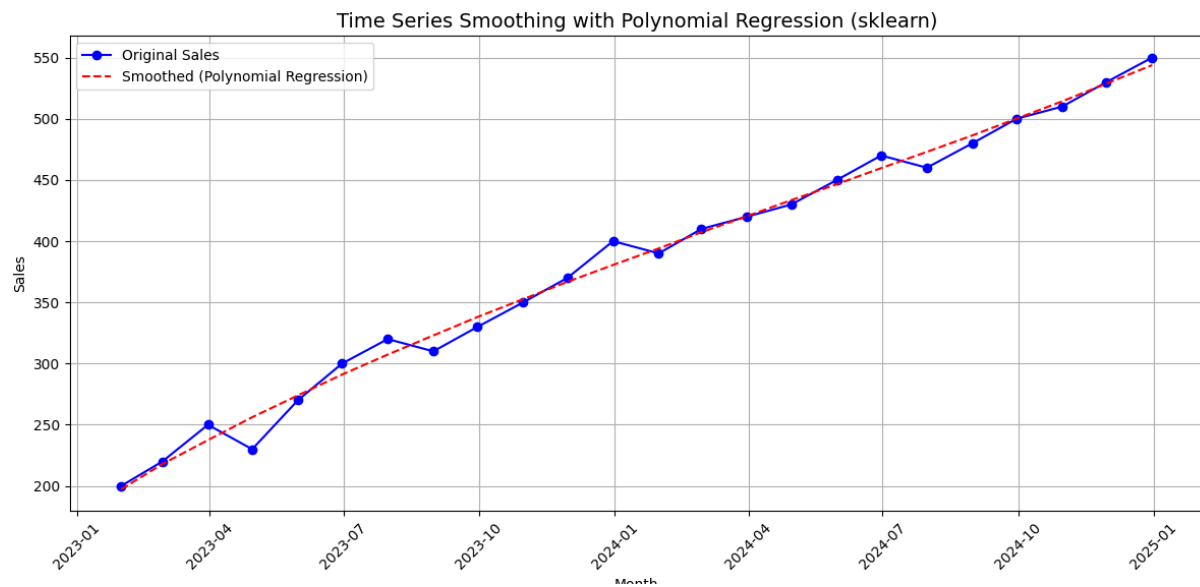
# Simulated monthly data
months = pd.date_range(start='2023-01', periods=24, freq='M')
sales = [200, 220, 250, 230, 270, 300, 320, 310, 330, 350, 370, 400,
         390, 410, 420, 430, 450, 470, 460, 480, 500, 510, 530, 550]

df = pd.DataFrame({'Month': months, 'Sales': sales})
df['TimeIndex'] = np.arange(len(df)) # numeric index for regression

# Polynomial Regression (degree=3)
X = df[['TimeIndex']]
y = df['Sales']
poly = PolynomialFeatures(degree=3)
X_poly = poly.fit_transform(X)

model = LinearRegression()
model.fit(X_poly, y)
df['Smoothed_Sales'] = model.predict(X_poly)

# Plotting
plt.figure(figsize=(12, 6))
plt.plot(df['Month'], df['Sales'], label='Original Sales', marker='o', color='blue')
plt.plot(df['Month'], df['Smoothed_Sales'], label='Smoothed (Polynomial Regression)', color='red', linestyle='--')
plt.title('Time Series Smoothing with Polynomial Regression (sklearn)', fontsize=14)
plt.xlabel('Month')
plt.ylabel('Sales')
plt.xticks(rotation=45)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```



### Explanation:

- We're using **PolynomialFeatures from sklearn** to create a nonlinear regression model that smooths the sales data.
- The degree=3 polynomial fits a smooth curve.
- This acts as a smoothing technique, especially useful when the time series has **non-linear trend components**.